

Tervezési minták a projektben

Az alábbi rövid összefoglaló három fontos tervezési mintát mutat be, és megmagyarázza, hogyan jelennek meg ezek a minták a te projektben. Minden minta esetén röviden leírom a problémát, a megoldást, és konkrét hivatkozást arra, hol látható a projektben.

DAO (Data Access Object) — Adat-hozzáférési objektum

Probléma: Az alkalmazásnak adatokat kell tárolnia és lekérnie (high score-ok, játékosok), de nem szeretnénk, hogy az üzleti logika közvetlenül SQL-lel vagy JDBC-vel legyen összekeverve.

Megoldás: A DAO egy réteget biztosít az adatbázis műveletekhez: a felsőbb rétegek (szolgáltatások, üzleti logika) a DAO felületét hívják, a DAO pedig végrehajtja a konkrét SQL műveleteket. Ez elválasztja a perzisztencia részleteit, könnyíti a tesztelést és a cserét (pl. más DB-re váltás).

Projekt példa

- **Osztályok:** *PlayerDao*, *DatabaseManager*, *PlayerRecord*.
- **Hol látszik:** *PlayerDao.upsertWin* és *PlayerDao.getHighScores* összegyűjtik és adják vissza a játékosok nyeréseit; a *HighScore* szolgáltatás ezeket a DAO metódusokat használja.
- **Előny:** a *HighScore* osztály nem ismeri a JDBC részleteit, csak a DAO felületét, így könnyebb mockolni tesztekhez.

Memento — Állapotmentés és visszaállítás

Probléma: Interaktív játékban szükség van a játék állapotának elmentésére és későbbi visszatöltésére anélkül, hogy a belső reprezentációt (táblák, belső mezők) szétszórta kezeljük.

Megoldás: A Memento minta lehetővé teszi egy objektum belső állapotának kinyerését és elmentését egy külső reprezentációba, majd későbbi visszaállítását. A mentés és visszatöltés logikája elkülönül a játék működésétől.

Projekt példa

- **Osztályok:** *Game* (mentés/betöltés), *Board* (belső állapot).
- **Hol látszik:** a *Game.saveGame(...)* és *Game.loadGame(...)* metódusok felelősek a játék állapotának fájlba írásáért és onnan olvasásáért; a fájl tartalmazza a játékos nevét, aktuális játékost, lépéseket és a tábla celláinak állapotát.

Strategy — Választható algoritmusok elválasztása

Probléma: Az AI lépéshozó logikája többféle stratégiát követhet (véletlen lépés, győzelem keresése, blokkolás). Ha ezeket a viselkedéseket egyetlen nagy metódusba kevernénk, nehéz lenne karbantartani és bővíteni.

Megoldás: A Strategy minta külön osztályokba zárja az algoritmusokat, és egy közös interfészen keresztül cserélhetővé teszi őket. A kliens (pl. játékvezérlés) dinamikusan kiválaszthatja a megfelelő stratégiát.

Projekt példa

- **Osztályok:** *GameService* (AI logika), *AIPlayer* (más játékhoz használható célzó logika).
- **Hol látszik:** a *GameService.placeNextAIMove* metódus több lépést próbál: *tryToWin*, *tryToBlockPlayer*, majd *tryRandomMove*. Ezek a külön logikai egységek funkcionálisan megfelelnek külön stratégiáknak; ha később külön *Strategy* interfészekre bontod őket, könnyen cserélhetők és tesztelhetők.